PROJECT PORTFOLIO · FULL-STACK ENGINEERING

School Management System.

A multi-tenant academic administration platform — accounts and roles, class and student lifecycles, timetable generation, and an arrival-based staff and student attendance engine — built for schools operating in Mogadishu, Somalia.

ROLE

Full-stack design & build

STACK

Node · Express · PostgreSQL · Prisma · React

CONTACT

ahmednornafic@gmail.com

12	31	27	1,145+
feature modules	database models	schema migrations	automated tests, passing

PURPOSE

One system, six kinds of people, one source of truth

Every school-day question — who's teaching what, who showed up, who's allowed to see which records — routes through a single, tenant-isolated platform instead of spreadsheets and side conversations.

The system is built for **schools, not a single school** — every account, class, timetable, and attendance record is scoped to exactly one tenant, with a platform-level Super Admin able to oversee schools without ever touching their day-to-day data. Inside a school, six roles share one data model but see six different slices of it, enforced independently at the route layer and the service layer, never trusted from the client alone.

Super Admin
Platform-wide oversight across every school

School Admin
Owns one school's configuration and staff

Manager
Runs day-to-day student & staff attendance

Teacher
Timetable-driven lesson attendance

Accountant
Own daily arrival, nothing more

Student
The subject of the records, never a recorder

ARCHITECTURE

Feature modules, not a monolith

Every domain area is its own vertical slice — routes, controller, service, repository, validation — with a narrow, explicit interface exposed to the rest of the system.

FRONTEND

React 18 + Vite

- Role-scoped routing
- Tailwind design system
- Optimistic, compact UI

→

API

Express feature modules

- JWT auth + RBAC gates
- Service-layer re-validation
- Shared audit & error handling

→

DATA

PostgreSQL + Prisma

- Per-school tenant scoping
- Partial unique indexes
- Full recorder/corrector audit fields

Defense in depth, deliberately. A role gate gets you past the route. Every write is re-validated in the service layer against the acting user's real assignment, role, and school — a Manager can no more spoof a Teacher's timetable slot than a client can spoof a server timestamp.

FEATURE TOUR

Twelve modules, one academic year at a time

From account creation to the last bell of the day — each module below is a self-contained backend feature with its own tests, paired with a matching frontend surface.

01

Authentication & Authorization

JWT sessions, password lifecycle, and a role matrix enforced as an allow-list — nothing is reachable by default.

02

Schools & Platform Management

Super Admin provisions schools; every other role's access is scoped to exactly one of them, permanently.

03

Classes & Sections

Grade levels, sections, and shift assignment — the roster backbone every other module reads from.

04

Students

Admission, enrollment, and lifecycle transitions — transfer, withdrawal, graduation — with history preserved, never deleted.

05

Teachers

Onboarding and self-service profile & preference management.

06

Subjects

A platform-wide catalog, assigned per school as each one actually teaches it.

07

Academic Years

Bounded years with a single designated "current" year every dated record anchors to.

08

Teacher Subject Assignments

Who teaches what, to which class, for which year — the join every timetable lesson is built from.

09

School Schedule

Shifts, periods, breaks, and the working-day calendar every other schedule concept is derived from.

10

Timetable Management

Automated generation with conflict detection and repair, draft/published versioning.

11

Weekly Subject Allocations

Target lesson counts per subject and grade — the constraint the generator satisfies.

12 · Case study below

Attendance Management

Student and staff attendance, arrival-based and self-correcting — the most iterated, most tested module in the system.

CASE STUDY

Attendance Management: getting the trust model right, twice

16 implementation phases, each traced back to a numbered Business Rule in a versioned Specification and Technical Design — every one triggered by a real product decision or a real, live-tested bug, never a guess.

Attendance sounds simple — present, absent, late — until six roles need six different slices of it, some of them able to record on someone else's behalf and some of them explicitly forbidden to. This module went through more revision than any other in the system, and every revision is traceable: a numbered Business Rule, a Technical Design section, an Implementation Plan phase, and a test that proves it.

v1.2

From manual status to derived status

Replaced a hand-picked Present/Absent/Late choice with an arrival-based model: a "Come" timestamp, and the server — never a human — derives Present, Late, or (past a configured cutoff) Absent.

v1.5

Killed the anti-pattern

Removed a design where a Manager clicked "Come" for every staff member individually — operationally unrealistic, and a weaker signal than genuine self-recording. Replaced with true self-check-in.

v1.6 – v1.7

Two bug reports, one confirmed, one disproved

Live-tested a claim that self check-in could be backdated — false, and hardened with regression tests anyway. Live-tested the *Excuse* action — confirmed it accepted any client-supplied date, including years in the past or future — and closed the gap with a today-only action plus a separate, explicitly-authorized historical-correction path.

v2.0

A narrower reversal, on purpose

Reintroduced Manager-recorded arrival for Teachers and Accountants — but scoped no wider than the Manager's existing correction authority: never another Manager, never an Admin, both paths sharing one uniqueness constraint.

v2.1

A timezone bug, root-caused before touched

A School Admin configured a 6 AM start time and got "not open yet" well past 6 AM local. Diagnosed read-only first: every comparison used raw UTC while the school runs on Africa/Mogadishu (UTC+3). Fixed with `Intl`-based timezone-aware comparisons — no new dependency, no change to what's stored, only to what "now" means.

> **The discipline that made this safe to iterate on:** every one of these changes was written up as a Business Rule before it was coded, and every fix started with reproducing the problem against a real running server and a real database — never a code read alone. The result is a module that's been rebuilt twice and still has zero regressions across 87 dedicated tests.

HOW I WORKED

Engineering practices, not just features

→ Documentation before code

Business Specification → Technical Design → Implementation Plan, in that order, for every feature and every amendment — versioned, changelogged, never skipped.

→ Investigate, then fix

Bug reports were reproduced against the live server and database before a line changed — twice, this overturned the assumption in the report itself.

→ Tenant isolation as a default

Every repository query is scoped by school; there is no query shape that can accidentally cross a tenant boundary.

→ Audit trails as two distinct facts

Who a record is *about* and who *recorded* it are always independently readable — never collapsed into one field, even when they're the same person.

→ Test against reality

1,145+ tests run as real HTTP requests against a real Express app and a real PostgreSQL database — not mocked service calls.

→ Scope discipline

Every change states explicitly what it does *not* touch — a habit that made a five-times-revised module safe to amend without regression.

School Management System

Full-stack project portfolio · Node.js, Express, PostgreSQL, Prisma, React, Tailwind CSS · ahmednornafic@gmail.com

`Node.js` `Express` `PostgreSQL` `Prisma ORM` `React 18` `React Router` `Vite`

Tailwind CSS

JWT Auth

Jest + Supertest